

Senet

Car Rental Webapp

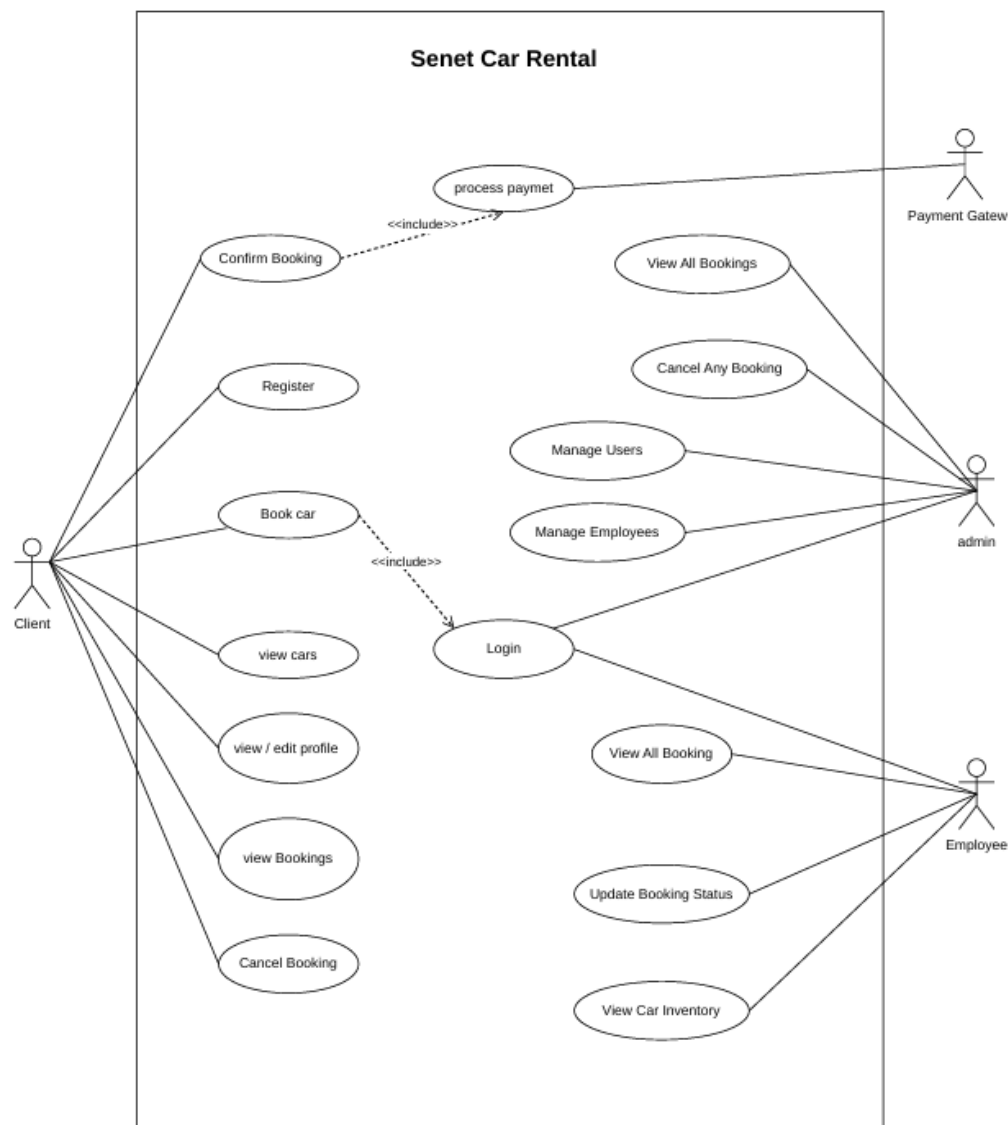
Document Type:	Software Requirements Specification(SRS) + OCL
Project:	SENET — Car Rental Webapp
Architecture:	Microservices + Spring Boot + Docker
Roles:	Client Employee Admin

1. Software Requirements Specification (SRS)

The SENET Webapp supports three roles: **Client** (browse & book cars), **Employee** (assist bookings, manage status, process refunds), and **Admin** (full system control). The following UML diagrams define the system's structure and behavior.

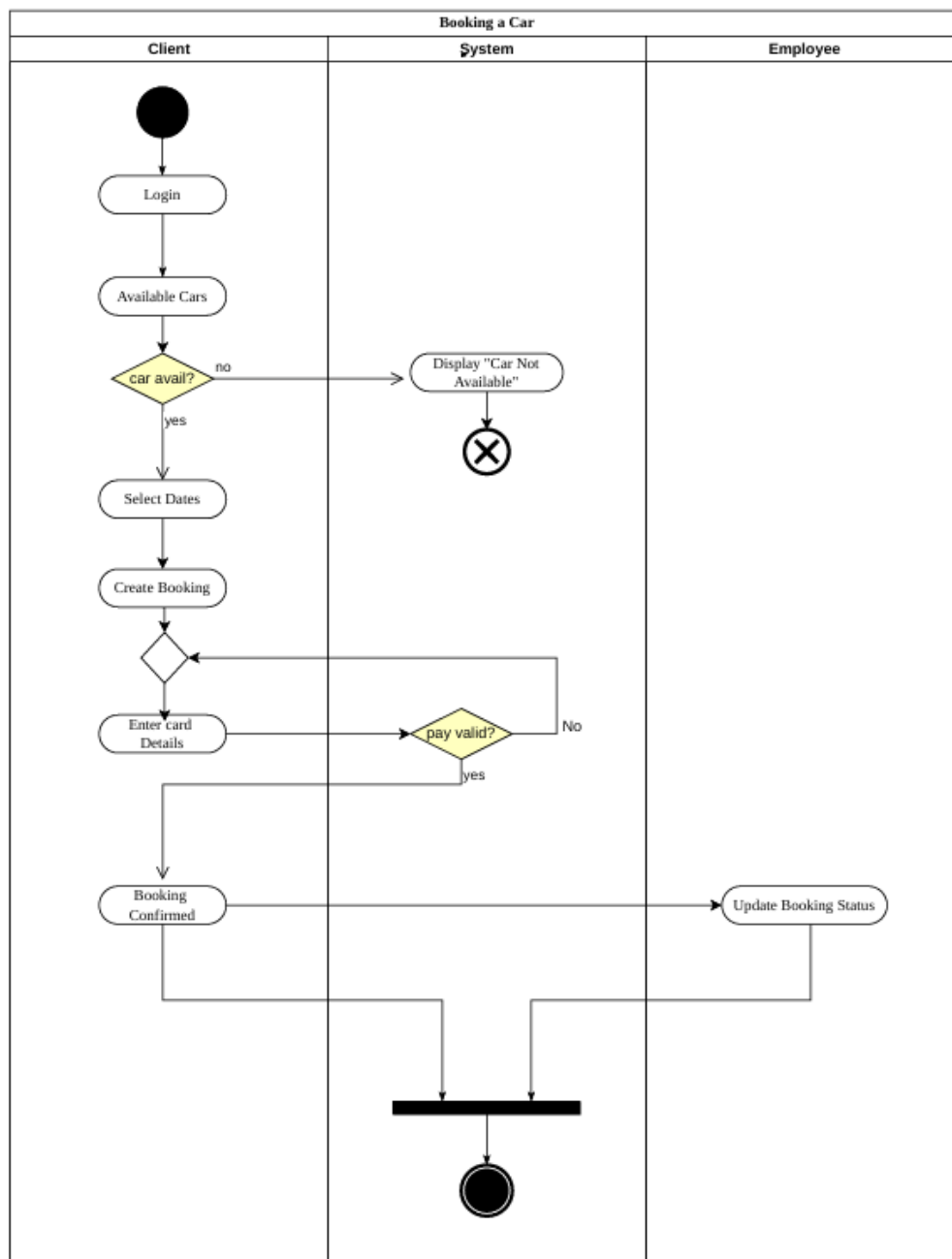
1.1 Use Case Diagram

The SENET Webapp involves four distinct actors. The **Client** performs self-service actions such as registration, browsing cars, and managing their profile and bookings. The **Employee** is responsible for operational tasks, including viewing all bookings, updating booking statuses, and managing car inventory. The **Admin** maintains full system control through CRUD operations over users, employees, and bookings. Additionally, the **Payment Gateway** acts as a secondary actor to facilitate the secure processing of payments, which is a mandatory step (included) during the booking confirmation flow.



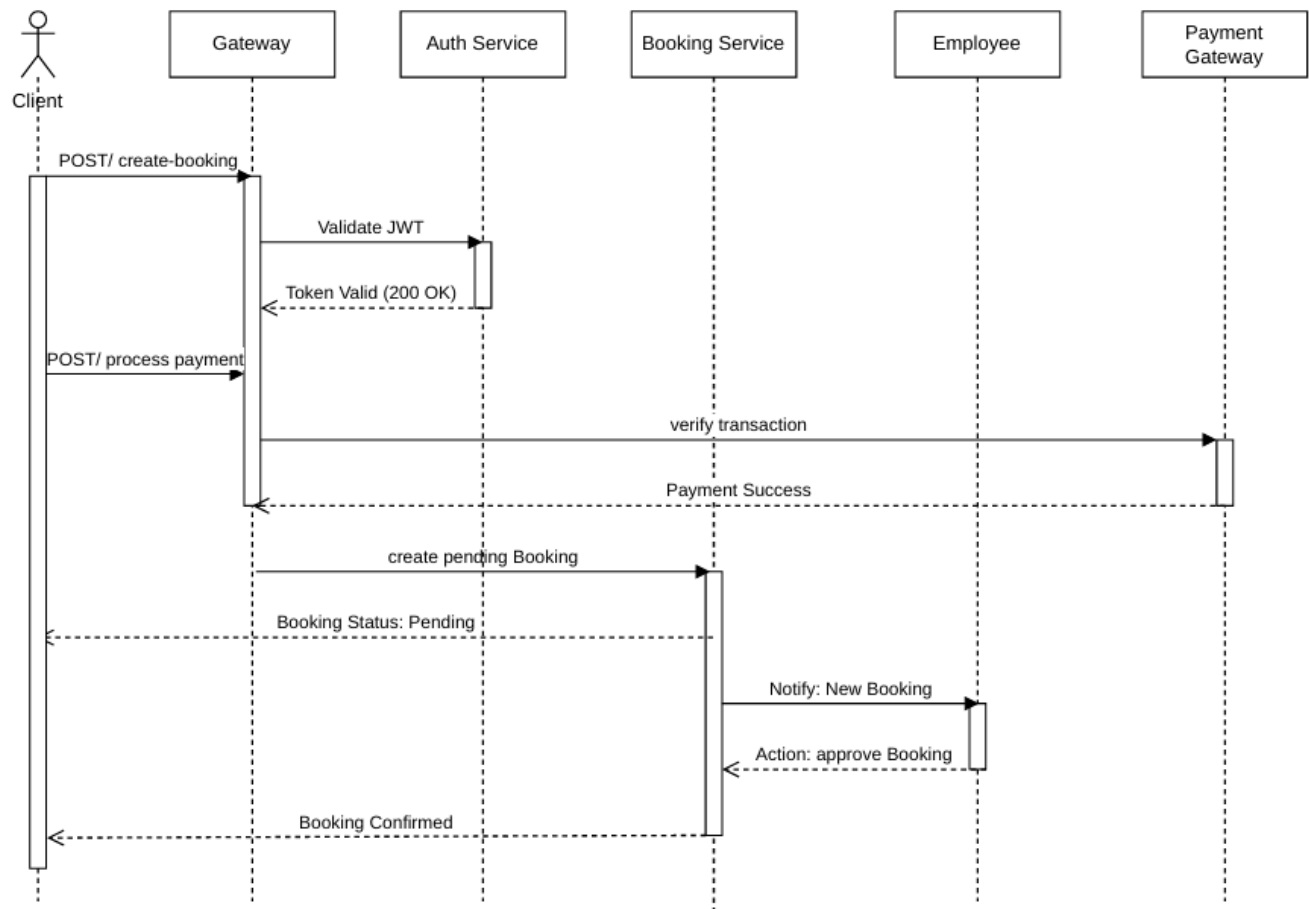
1.2 Activity Diagram

This Activity Diagram outlines the booking workflow across three lanes: **Client**, **System**, and **Employee**. The process begins with the Client's selection and a system availability check. If valid, the System handles payment verification and persistence. Finally, the flow concludes with the Employee updating the booking status, ensuring all roles are synchronized before the process terminates at the final node.



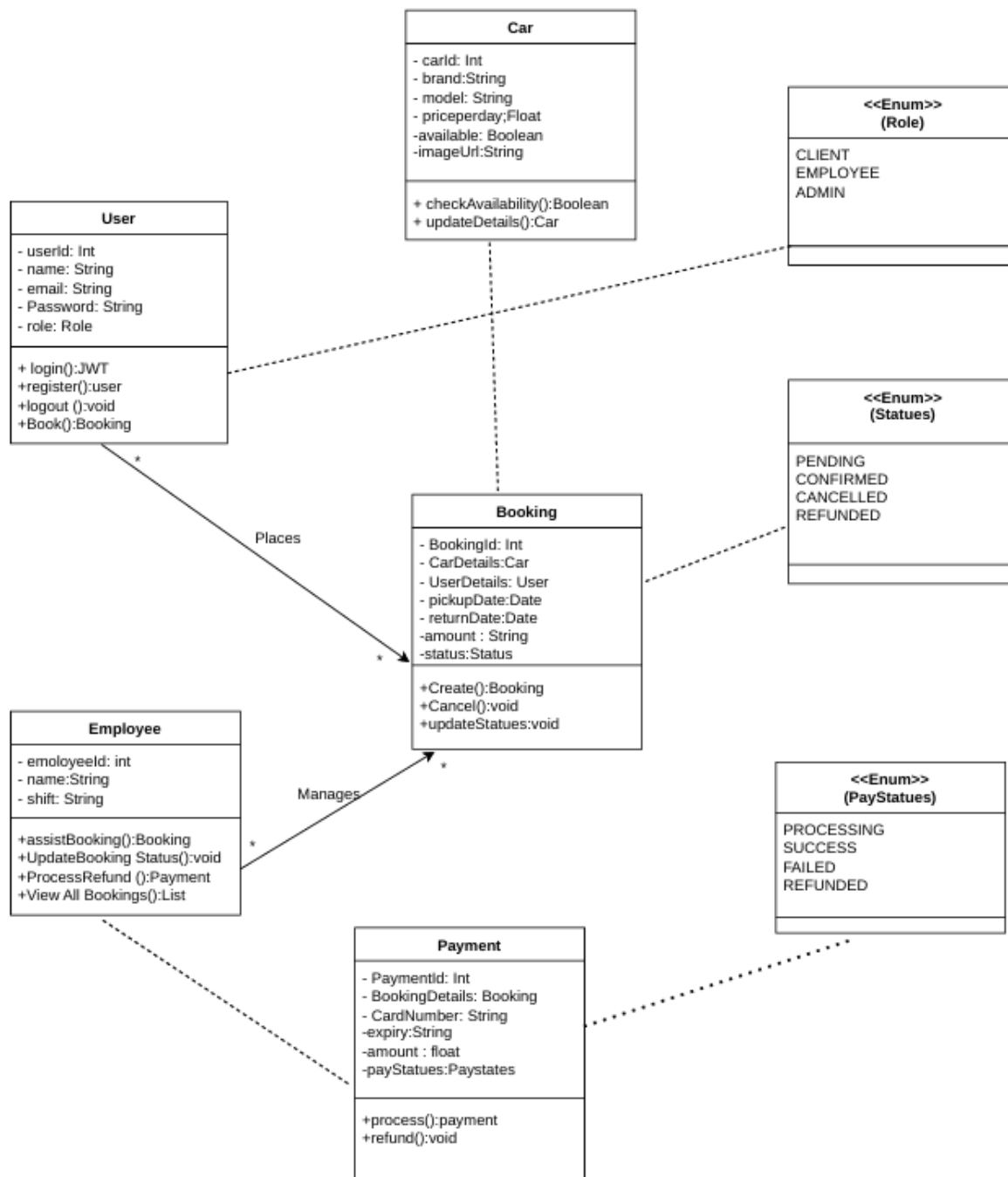
1.3 Sequence Diagram

This diagram illustrates the car booking flow. First, the Gateway validates the Client's JWT via the **Auth Service**. After authentication, the Client completes the payment through the **Payment Gateway**, creating a "Pending Approval" record. Finally, the Employee reviews and approves the booking, triggering the Booking Service to confirm the reservation and notify the Client.



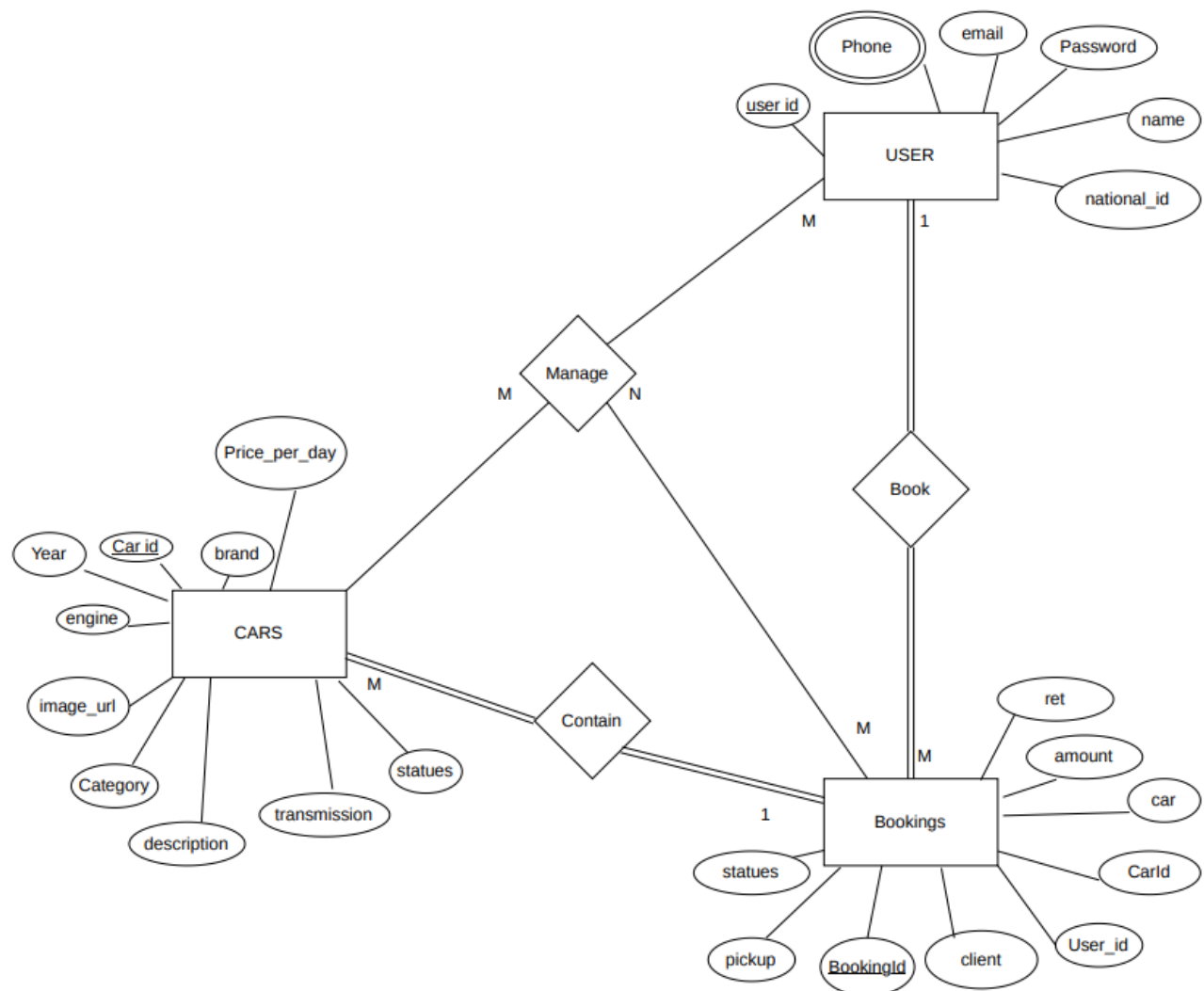
1.4 Class Diagram

This Class Diagram defines the system's structural entities. It consists of **five core domain classes**: **User**, **Employee**, **Car**, **Booking**, and **Payment**. The Employee class manages booking records, while the User places them. The system's logic is supported by three enumerations: Role (defining access levels), Status (tracking booking progress), and PayStatues (monitoring payment states).



1.5 Entity-Relationship Diagram (ERD)

This ER Diagram maps the database structure for the SENET Webapp, featuring three primary entities: **USER**, **CARS**, and **Bookings**. The system enforces role-based access control through its relationships: if a user is a **Client**, they are restricted to the "Book" relationship to create reservations. However, if the user is an **Admin** or **Employee**, they transition to the "Manage" relationship, granting them the authority to oversee car inventory and process booking records. This structure ensures that functional capabilities are strictly aligned with the user's designated role.



2. Object Constraint Language (OCL)

2.1 Context: Car

Rule: Car price must be positive

```
context Car inv PriceIsPositive: self.pricePerDay > 0
```

Rule: Car must have non-empty brand and model

```
context Car inv HasBrandAndModel: self.brand <> "" and self.model <> ""
```

2.2 Context: User

Rule: Email must be unique across all users

```
context User inv UniqueEmail: User.allInstances()->isUnique(u | u.email)
```

Rule: User name must not be empty

```
context User inv NameNotEmpty: self.name <> ""
```

2.3 Context: Employee

Rule: Employee must be linked to a valid User with role=EMPLOYEE

```
context Employee inv ValidUserRole: self.user.role = Role::EMPLOYEE
```

Rule: An employee cannot be their own manager

```
_context Employee inv CannotSelfManage: self.managerId <> self.employeeId
```

Rule: Employee can only handle bookings with status PENDING or CONFIRMED

```
context Employee inv HandlesActiveBookings: self.assistedBookings->forAll(b | b.status =  
Status::PENDING or b.status = Status::CONFIRMED)
```

2.4 Context:Booking

Rule: Return date must be strictly after pickup date

```
context Booking inv ReturnAfterPickup: self.returnDate > self.pickupDate
```

Rule: Booking amount must be positive

```
context Booking inv AmountIsPositive: self.amount > 0
```

Rule: A CONFIRMED booking requires a successful

```
payment context Booking inv ConfirmedRequiresPayment: self.status = Status::CONFIRMED implies  
self.payment.payStatus = PayStatus::SUCCESS
```

2.5 Context:Payment

Rule: Payment amount must match its booking amount

```
context Payment inv AmountMatchesBooking: self.amount = self.booking.amount
```

Rule: Payment amount must be positive

```
context Payment inv AmountIsPositive: self.amount > 0
```

Rule: Card expiry must be exactly 5 characters (MM/YY)

```
context Payment inv ValidExpiry: self.expiry <> "" and self.expiry.size() = 5
```

2.6 Context: Operation — Booking::create()

Rule: Pre: if employeeId provided, employee must exist and be active

```
context Booking::create(carId:Long, clientId:Long, pickupDate:LocalDate, returnDate:LocalDate,  
employeeId:Long) pre ValidEmployee: employeeId <> null implies Employee.allInstances()->exists(e |  
e.employeeId = employeeId)
```


Rule: Post: booking status is PENDING after creation

context Booking::create(carId:Long, clientId:Long, pickupDate:LocalDate, returnDate:LocalDate, employeeId:Long) post StatusIsPending: result.status = Status::PENDING

OCL Constraint Summary

Context	Constraint Name	Type
Car	PricelsPositive	Invariant
Car	HasBrandAndModel	Invariant
Car	AvailableForBooking	Invariant
User	UniqueEmail	Invariant
User	NameNotEmpty	Invariant
Employee	ValidUserRole	Invariant
Employee	CannotSelfManage	Invariant
Employee	HandlesActiveBookings	Invariant
Booking	AmountIsPositive	Invariant
Payment	AmountIsPositive	Invariant
Booking::create	CarAvailable	Precondition
Booking::create	StatusIsPending	Postcondition
Booking::cancel	AuthorizedToCancel	Precondition
Booking::cancel	CancellableStatus	Precondition
Booking::cancel	StatusIsCancelled	Postcondition
Booking::cancel	CarAvailableAgain	Postcondition